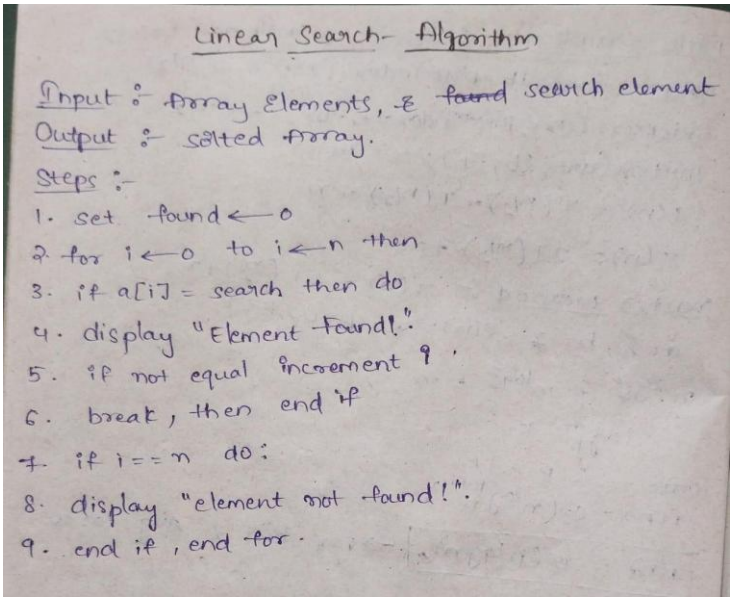
PRACTICAL - 2

AIM : To implementation and Time analysis of linear and binary search algorithm.

LINEAR SEARCH –

ALGORITHM:



PROGRAM:

```
#include <iostream> using namespace std;

int main()
{
   int arr[100], i, n, key;

   cout << "Enter number of array elements: " << endl;     cin >> n;

   cout << "Enter array elements: " << endl;     for (i = 0; i < n; i++)
   {        cin >> arr[i];
   }

   cout << "Enter element to search: " << endl;     cin >> key;
```

```
    for (i = 0; i < n; i++)
    {       if (arr[i] == key)
       {
          cout << "Element found at index: " << i << endl;          return 0;
       }
    }

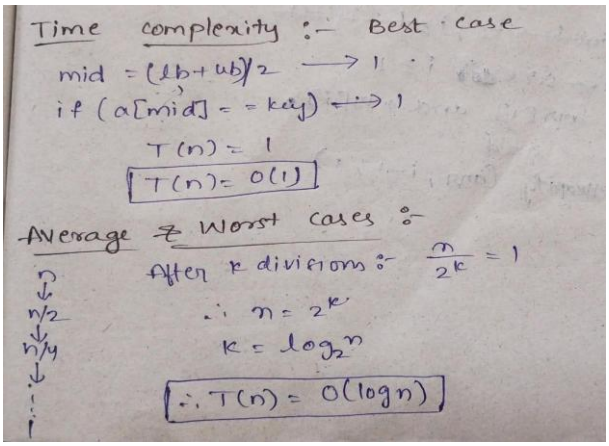
    cout << "Element not found." << endl;

    return 0;
}
```

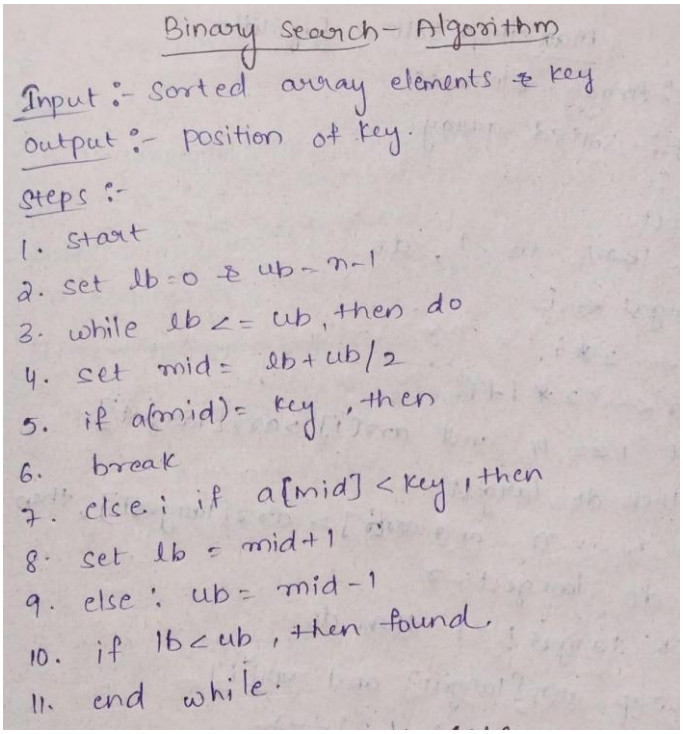
OUTPUT:

```
enter number of array elements:
5
enter array elements:
7
9
5
34
78
enter element to serach:
5
element found at index of:2
```

TIME COMPLEXITY:



2. BINARY SEARCH - ALGORITHM:


Binary Search - Algorithm

Input :- Sorted array elements & key
output :- position of key.
steps :-
1. start
2. set lb = 0 & ub = n-1
3. while lb <= ub, then do
4. set mid = lb + ub / 2
5. if a(mid) = key, then
6. break
7. else if a[mid] < key, then
8. set lb = mid + 1
9. else : ub = mid - 1
10. if lb < ub, then found.
11. end while.

PROGRAM:

```
#include <iostream> using namespace std;

int main() {     int arr[100], n, key;     int low, high, mid;

   cout << "Enter number of array elements: " << endl;     cin >> n;

   cout << "Enter array elements in sorted order: " << endl;     for (int i = 0; i
< n; i++)
   {
      cin >> arr[i];
   }

   cout << "Enter element to search: " << endl;     cin >> key;

   low = 0;     high = n - 1;
```

```
    while (low <= high)
    {       mid = low + (high - low) / 2;

       if (arr[mid] == key)
       {
          cout << "Element found at index: " << mid << endl;          return 0;
       }
       else if (arr[mid] < key)
       {           low = mid + 1;
       }       else       {           high = mid - 1;
       }
    }

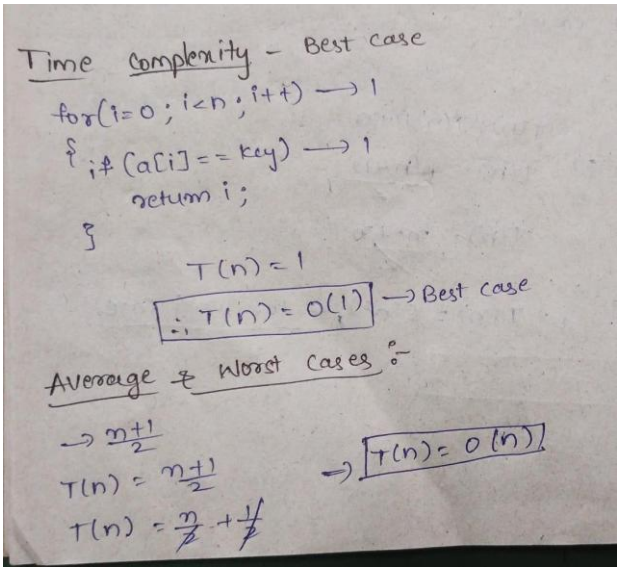
    cout << "Element not found." << endl;

    return 0;
}
```

Output:

```
Enter number of array elements: 6
Enter array elements: 8
4
6
1
9
19
Sorted elements: 1 4 6 8 9 19
Enter element to search: 6
Element found at index 2
```

TIME COMLEXITY:



CONCLUSION:

The practical successfully implemented and analyzed Linear Search and Binary Search algorithms. Linear Search checks each element sequentially and can be used on both sorted and unsorted data, whereas Binary Search repeatedly divides a sorted array into two halves to locate the required element.

From the time analysis, Linear Search has a best-case complexity of O(1) and a worst-case complexity of O(n), while Binary Search has a best-case complexity of O(1) and a worst-case complexity of O(log n). Therefore, Binary Search is more efficient for large sorted datasets. This practical helped in understanding the working, implementation, and performance differences between the two searching techniques.